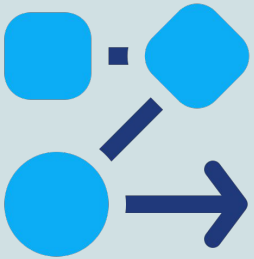
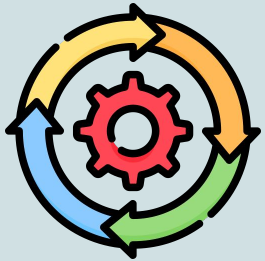
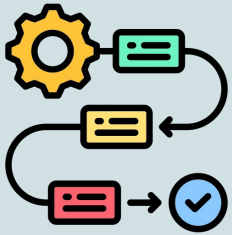
Lesson 4

Control Flows, Loops, Methods







Control Flow I:

`if`/`else` & Guard Clauses

What Is Control Flow?

Control flow decides **which part of the code runs** based on certain conditions.

Syntax:

```
if (condition) {
    // code executes if condition is true
}
```

What Is Control Flow?

Control flow decides **which part of the code runs** based on certain conditions.

Syntax:

```
if (condition) {
    // code executes if condition is true
}
```

Example:

```
int age = 18;
if (age >= 18) {
    System.out.println("You can vote!");
}
```

You can vote!

```
int temperature = 15;

if (temperature > 20) {
    System.out.println("It’s a warm day!");
} else {
    System.out.println("It’s a cold day!");
}
```

It’s a cold day!

What Are Guard Clauses?

A **guard clause** is an early exit from a method or function.
Instead of nesting `if`s, we *guard* the main logic with **quick returns** for invalid conditions.

Bad Practice ❌

```
public void processOrder(Order order) {
    if (order != null) {
        if (order.isPaid()) {
            System.out.println("Processing order...");
        } else {
            System.out.println("Order not paid");
        }
    } else {
        System.out.println("Order is null");
    }
}
```

Good Practice ✅

```
public void processOrder(Order order) {
    if (order == null) {
        System.out.println("Order is null");
        return;
    }

    if (!order.isPaid()) {
        System.out.println("Order not paid");
        return;
    }

    System.out.println("Processing order...");
}
```

What Are Guard Clauses?

A **guard clause** is an early exit from a method or function.
Instead of nesting `if`s, we *guard* the main logic with **quick returns** for invalid conditions.

Bad Practice ❌

```
public void makePayment(User user, double amount) {
    if (user != null) {
        if (amount > 0) {
            if (user.hasBalance(amount)) {
                System.out.println("Payment successful!");
            } else {
                System.out.println("Not enough balance");
            }
        } else {
            System.out.println("Invalid amount");
        }
    } else {
        System.out.println("User not found");
    }
}
```

Good Practice ✅

```
public void makePayment(User user, double amount) {
    if (user == null) {
        System.out.println("User not found");
        return;
    }

    if (amount <= 0) {
        System.out.println("Invalid amount");
        return;
    }

    if (!user.hasBalance(amount)) {
        System.out.println("Not enough balance");
        return;
    }

    System.out.println("Payment successful!");
}
```

Java Ternary Operator

The **ternary operator** is a **shortcut for an if-else statement** it lets you write **conditional logic in a single line**.

Syntax:

```
variable = (condition) ? valueIfTrue : valueIfFalse;
```

Java Ternary Operator

The **ternary operator** is a **shortcut for an if-else statement** it lets you write **conditional logic in a single line**.

Syntax:

```
variable = (condition) ? valueIfTrue : valueIfFalse;
```

Example:

```
String result = null;
if (age >= 18) {
    result = "Adult";
} else {
    result = "Minor";
}
System.out.println(result);
```

Java Ternary Operator

The **ternary operator** is a **shortcut for an if-else statement** it lets you write **conditional logic in a single line**.

Syntax:

```
variable = (condition) ? valueIfTrue : valueIfFalse;
```

Example:

```
String result = null;
if (age >= 18) {
    result = "Adult";
} else {
    result = "Minor";
}
System.out.println(result);
```

```
int age = 20;
String result = (age >= 18) ? "Adult" : "Minor";
System.out.println(result);
```

Control Flow II:
Modern switch (arrow & yield)

What Is a switch Statement?

In Java, a switch helps you **replace long if-else if chains** with cleaner and faster decision logic.

Example:

```
String day = "MONDAY";
switch (day) {
    case "MONDAY":
        System.out.println("Start of the week!");
        break;
    case "FRIDAY":
        System.out.println("Weekend is near!");
        break;
    default:
        System.out.println("Midweek day");
}
```

What Is a switch Statement?

In Java, a switch helps you **replace long if-else if chains** with cleaner and faster decision logic.

Example:

```
String day = "MONDAY";
switch (day) {
    case "MONDAY":
        System.out.println("Start of the week!");
        break;
    case "FRIDAY":
        System.out.println("Weekend is near!");
        break;
    default:
        System.out.println("Midweek day");
}
```

Modern Swithc

It’s **cleaner**, **safer**, and can even **return values**.

Syntax:

```
switch (variable) {
    case VALUE1 -> statement;
    case VALUE2 -> statement;
    default -> statement;
}
```

Modern Swithc

It's **cleaner**, **safer**, and can even **return values**.

Syntax:

```
switch (variable) {
    case VALUE1 -> statement;
    case VALUE2 -> statement;
    default -> statement;
}
```

Example:

```
String day = "MONDAY";

switch (day) {
    case "MONDAY" -> System.out.println("Start of the week!");
    case "FRIDAY" -> System.out.println("Weekend is coming!");
    default -> System.out.println("Midweek grind!");
}
```

Modern Swithc

It's **cleaner**, **safer**, and can even **return values**.

Syntax:

```
switch (variable) {
    case VALUE1 -> statement;
    case VALUE2 -> statement;
    default -> statement;
}
```

Example:

```
String day = "MONDAY";

switch (day) {
    case "MONDAY" -> System.out.println("Start of the week!");
    case "FRIDAY" -> System.out.println("Weekend is coming!");
    default -> System.out.println("Midweek grind!");
}
```

Modern Swithc

It's **cleaner**, **safer**, and can even **return values**.

Syntax:

```
switch (variable) {
    case VALUE1 -> statement;
    case VALUE2 -> statement;
    default -> statement;
}
```

Example:

```
String day = "MONDAY";

switch (day) {
    case "MONDAY" -> System.out.println("Start of the week!");
    case "FRIDAY" -> System.out.println("Weekend is coming!");
    default -> System.out.println("Midweek grind!");
}
```

Using switch as an Expression

Now switch can **return a value** directly.

Example:

```
String day = "FRIDAY";

String message = switch (day) {
    case "MONDAY" -> "Start of the week!";
    case "FRIDAY" -> "Weekend vibes!";
    default -> "Just another day.";
};

System.out.println(message);
```

Multi-label Cases

You can handle multiple values in one line.

Example:

```
String day = "SATURDAY";

String type = switch (day) {
    case "SATURDAY", "SUNDAY" -> "Weekend";
    case "MONDAY", "TUESDAY", "WEDNESDAY", "THURSDAY", "FRIDAY" -> "Weekday";
    default -> "Unknown";
};

System.out.println(type);
```

Using yield (for Multi-line Logic)

When you have **multiple statements** in a case, you can use **yield** to return a value.

Example:

```
int marks = 85;

String grade = switch (marks / 10) {
    case 10, 9 -> "A";
    case 8 -> {
        System.out.println("Good performance!");
        yield "B";
    }
    case 7 -> {
        System.out.println("Needs improvement.");
        yield "C";
    }
    default -> "F";
};

System.out.println("Grade: " + grade);
```

Using yield (for Multi-line Logic)

When you have **multiple statements** in a case, you can use **yield** to return a value.

Example:

```
int marks = 85;

String grade = switch (marks / 10) { // 8
    case 10, 9 -> "A";
    case 8 -> {
        System.out.println("Good performance!");
        yield "B";
    }
    case 7 -> {
        System.out.println("Needs improvement.");
        yield "C";
    }
    default -> "F";
};

System.out.println("Grade: " + grade);
```

Using yield (for Multi-line Logic)

When you have **multiple statements** in a case, you can use **yield** to return a value.

Example:

```
int marks = 85;

String grade = switch (marks / 10) { // 8
    case 10, 9 -> "A";
    case 8 -> {
        System.out.println("Good performance!");
        yield "B";
    }
    case 7 -> {
        System.out.println("Needs improvement.");
        yield "C";
    }
    default -> "F";
};

System.out.println("Grade: " + grade); // B
```

Loops I:

for, while, do-while

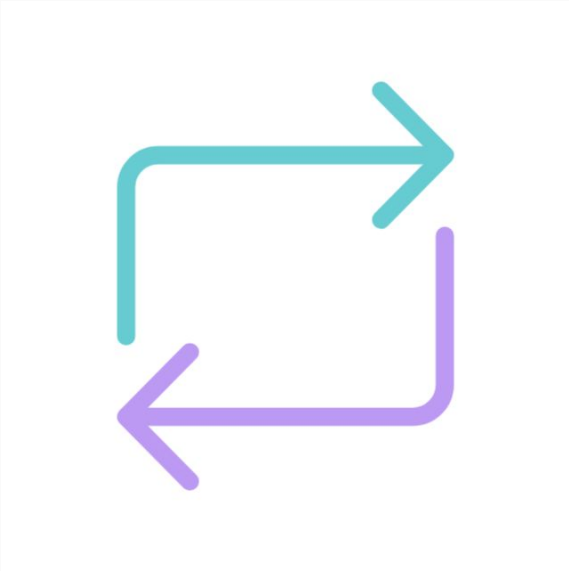
when to use which

What Are Loops?

A loop is used when you need to repeat an action several times.

For example:

- Print ‘Hello’ 10 times
- Show all student names

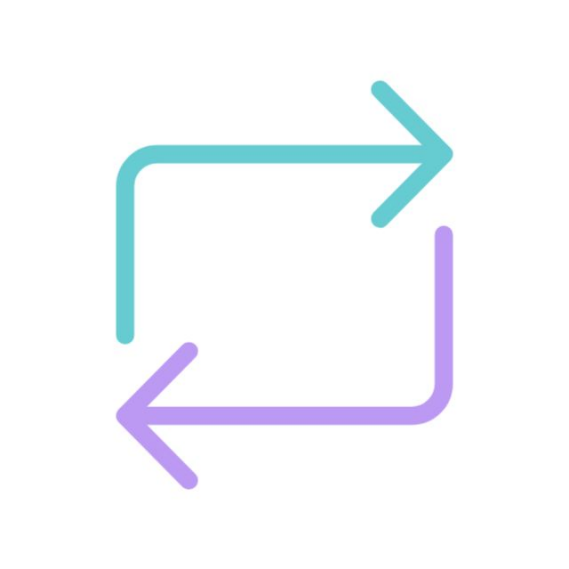


What Are Loops?

A loop is used when you need to repeat an action several times.

For example:

- Print ‘Hello’ 10 times
- Show all student names

In Java (and most programming languages), there are three main types of loops:

- **for**
- **while**
- **do-while**